Lecture 14: Multi-Client TCP Servers

Objectives:

- Learn how to create multi-client TCP servers using the **Thread** class
- Learn how to create multi-client TCP servers using the **ThreadPool** class
- Learn how to create multi-client TCP servers using **asynchronous methods**

Introduction

In UDP, since there is no connection involved, it is trivial to create a multi-client server as we saw in the UDP client-server programming lab.

However, in the case of TCP, especially where connection must be maintained throughout a session, a multi-client server can only be achieved through the use of threads or asynchronous methods.

In this lecture, we look at there possible ways of creating a multi-client TCP server in C#.

1. Creating Multi-Client Servers using the Thread class

So far we have used Threads to provide an efficient communication in a single client system and to build responsive GUI applications.

We can also use threads to allow a server to maintain more than one session – with multiple clients, at the same time.

To a achieve this, the server application is partition into two parts, namely, connection acceptor and connection handler.

The connection acceptor is the main application which runs in an infinite loop waiting for a client to connect.

Each time a client connects; the "connection acceptor" accepts the client and then performs the following:

- creates an instance of the "connection handler"
- creates a separate thread, associating it with the connection handler instance and

- starts the thread.

The connection handler instance is responsible for communicating with the client throughout the session.

The connection handler should be a separate class, which has a method that matches the signature of the **ThreadStart** delegate:

void method()

Example 1:

The following example shows a multi-client echo client-server system.

```
class MultiThreadedTcpServer
{
     public static void Main(string[] args)
     {
          int port = 9070;
          Socket server = new Socket(AddressFamily.InterNetwork,

SocketType.Stream,
                                        ProtocolType.Tcp);
          IPEndPoint endpoint = new IPEndPoint(IPAddress.Any, port);
          server.Bind(endpoint);

          server.Listen(10);
          Console.WriteLine("Waiting for clients on port " + port);

          while(true) {
               try {
                    Socket client = server.Accept();
                    ConnectionHandler handler = new
ConnectionHandler(client);

                    Thread thread = new Thread(new

ThreadStart(handler.HandleConnection));
                    thread.Start();
               } catch(Exception) {
                    Console.WriteLine("Connection failed on port
"+port);
               }
          }
     }
}

class ConnectionHandler {

     private Socket client;
     private NetworkStream ns;
     private StreamReader reader;
     private StreamWriter writer;
```

```
     private static int connections = 0;

     public ConnectionHandler(Socket client) {
          this.client = client;
     }

     public void HandleConnection() {
          try
          {
               ns = new NetworkStream(client);
               reader = new StreamReader(ns);
               writer = new StreamWriter(ns);
               connections++;
               Console.WriteLine("New client accepted: {0} active
connections", connections);

               writer.WriteLine("Welcome to my server");
               writer.Flush();

               string input;
               while(true) {
                    input = reader.ReadLine();
                    if (input.Length == 0 || input.ToLower() == "exit")
                         break;

                    writer.WriteLine(input);
                    writer.Flush();
               }
               ns.Close();
               client.Close();
               connections--;
               Console.WriteLine("Client disconnected: {0} active
connections", connections);
          } catch(Exception) {
               connections--;
               Console.WriteLine("Client disconnected: {0} active
connections", connections);
          }
     }
}
```

The above server can be tested using any of the Tcp echo client discussed earlier.

2. Creating Multi-Client Servers using the ThreadPool class

Firing a thread to handle each client without any control as we did in the above example is a sure way to crash a system.

It should be understood that in most cases it is one processor that is being shared among all the threads. Obviously, the more threads, the less the time allocated for each thread, hence, the slower the application.

To help avoid this problem and also to void the overhead in creating and deleting threads, C# provides the **ThreadPool** class.

The **ThreadPool** class provides a set of reusable threads, maintained by the system, which can be used to assigned tasks that is normally done by user threads.

ThreadPool allows a maximum of 25 threads. Any request for a thread is enqueued until threads become available.

The following are some of the methods of the **ThreadPool** class. All the methods are static:

GetAvailableThreads	Retrieves the difference between the maximum number of thread pool threads, returned by GetMaxThreads, and the number currently active.
GetMaxThreads	Retrieves the number of requests to the thread pool that can be active concurrently. All requests above that number remain queued until thread pool threads become available.
QueueUserWorkItem	Overloaded. Queues a method for execution. The method executes when a thread pool thread becomes available.

Exampe 2:
The following example modifies example 1 above to use ThreadPool.

```
public static void Main(string[] args)
{
     int port = 9070;
     Socket server = new Socket(AddressFamily.InterNetwork,
SocketType.Stream,
                                        ProtocolType.Tcp);
     IPEndPoint endpoint = new IPEndPoint(IPAddress.Any, port);
     server.Bind(endpoint);
     server.Listen(10);
     Console.WriteLine("Waiting for clients on port " + port);

     while(true) {
          try {
               Socket client = server.Accept();
```

```
                ConnectionHandler handler = new ConnectionHandler(client);

                ThreadPool.QueueUserWorkItem(new
WaitCallback(handler.HandleConnection));
            } catch(Exception) {
                Console.WriteLine("Connection failed on port "+port);
            }
        }
}
```

```
    public void HandleConnection(Object state) {...}
```

Notice the change in the signature of the HandleConnection method.

3. Creating multi-client system using asynchronous methods

The third option in creating multi-client application is by using asynchronous methods as shown below.

Example 3:

The following example uses asynchronous socket methods to implement a windows-based asynchronous TCP echo client and server.

```
public class MultiClientAsyncEchoServer : System.Windows.Forms.Form {
    private System.Windows.Forms.Label label3;
    private System.Windows.Forms.Label label2;
    private System.Windows.Forms.Label label;
    private System.Windows.Forms.TextBox statusBox;
    private System.Windows.Forms.Button startServer;
    private System.Windows.Forms.TextBox txtPort;
    private System.Windows.Forms.ListBox resultBox;

    private Socket server;
    private byte[] data = new byte[1024];
    private int connections = 0;

    public MultiClientAsyncEchoServer(){
        InitializeComponent();
    }

    void startServerClick(object sender, System.EventArgs e)    {
        server = new Socket(AddressFamily.InterNetwork,
SocketType.Stream,
                            ProtocolType.Tcp);
        int port = int.Parse(txtPort.Text);
        IPEndPoint localEP = new IPEndPoint(IPAddress.Any, port);
        server.Bind(localEP);
```

```
            server.Listen(4);
            startServer.Enabled = false;
            server.BeginAccept(new AsyncCallback(OnConnected), server);

      }


      void OnConnected(IAsyncResult result)   {
            Socket client = server.EndAccept(result);
            connections++;
            server.BeginAccept(new AsyncCallback(OnConnected), server);
            try   {
                  statusBox.Text = ""+connections;
                  byte[] message = Encoding.ASCII.GetBytes("Welcome to my
Server");
                  client.BeginSend(message, 0, message.Length,
SocketFlags.None,
                                    new AsyncCallback(OnDataSent), client);
            }
            catch(SocketException)   {
                  CloseClient(client);
            }
      }

      void OnDataSent(IAsyncResult result) {
            Socket client = (Socket) result.AsyncState;
            try   {
                  int sent = client.EndSend(result);
                  client.BeginReceive(data, 0, data.Length,
SocketFlags.None,
                                          new AsyncCallback(OnDataReceived),
client);
            }
            catch(SocketException) {
                  CloseClient(client);
            }
      }

      void OnDataReceived(IAsyncResult result){
            Socket client = (Socket) result.AsyncState;
            try   {
                  int receive = client.EndReceive(result);
                  if (receive == 0) {
                        CloseClient(client);
                        return;
                  }
                  else {
                        string message = Encoding.ASCII.GetString(data, 0,
receive);
                        resultBox.Items.Add(message);
                        byte[] echoMessage =
Encoding.ASCII.GetBytes(message);
                        client.BeginSend(echoMessage, 0, echoMessage.Length,
```

```
SocketFlags.None,
                                        new AsyncCallback(OnDataSent), client);
                    }
               }
               catch(SocketException) {
                    CloseClient(client);
               }
          }

          public void CloseClient(Socket client) {
               client.Close();
               connections--;
               statusBox.Text = ""+connections;
          }

          public static void Main() {
               Application.Run(new MultiClientAsyncEchoServer());
          }

          void InitializeComponent() {
          }
}
```